



Fig Format 3.2

This is a description of the *Fig Format 3.2*, which is a copy of xfig.3.2.x/Doc/FORMAT3.2 in the **xfig 3.2.x** source distribution.

```
/*
 * FIG : Facility for Interactive Generation of figures
 * Copyright (c) 1985 by Supoj Sutanthavibul
 * Parts Copyright (c) 1989-2002 by Brian V. Smith
 * Parts Copyright (c) 1991 by Paul King
 * Parts Copyright (c) 1995 by C. Blanc and C. Schlick
 *
 * Any party obtaining a copy of these files is granted, free of charge, a
 * full and unrestricted irrevocable, world-wide, paid up, royalty-free,
 * nonexclusive right and license to deal in this software and
 * documentation files (the "Software"), including without limitation the
 * rights to use, copy, modify, merge, publish and/or distribute copies of
 * the Software, and to permit persons who receive copies from any such
 * party to do so, with the only requirement being that this copyright
 * notice remain intact.
 */
```

The new components in protocol 3.2 are the paper size, magnification, single/multiple page indicator and transparent color for GIF export in the header.

The other modification between version 3.1 and version 3.2 of the protocol is the mathematical model used for splines. The new version uses X-splines which allows the user to mix interpolation and approximation points in a same curve. More precisely, it means that an X-spline curve is neither an interpolated spline nor an approximated one, it is BOTH (the behaviour of each point is controlled by one single parameter called "shape factor"). For additional information about X-splines, see:

"X-Splines: A Spline Model Designed for the End User"
by C. Blanc and C. Schlick, Proceedings of SIGGRAPH'95

Caveat: Because spline models of previous versions (quadratic B-splines and Bezier with hidden points) are no longer supported, curves that are present in version 3.1 and older files are automatically converted to X-splines. This translation is only an approximation process. It means that the converted curves are not exactly the same as the original ones. Though the translation usually provides almost identical curves, some hand-fitting may be needed in some pathological cases.

Description of the Fig Format Follows

(1) The very first line is a comment line containing the name and version:
#FIG 3.2

The character # at the first column of a line indicates that the line is a comment line which will be preserved when the Fig file is read in.

The user may edit them with the popup editor.

The comment line(s) must immediately precede the object to which they are associated. In the case of the "whole figure comments" mentioned below, they immediately precede the (resolution,coord_system) line.

(2) The first non-comment line consists of the following:

```
string  orientation      ("Landscape" or "Portrait")
string  justification    ("Center" or "Flush Left")
string  units            ("Metric" or "Inches")
string  papersize        ("Letter", "Legal", "Ledger", "Tabloid",
                          "A", "B", "C", "D", "E",
                          "A4", "A3", "A2", "A1", "A0" and "B5")

float   magnification    (export and print magnification, %)
string  multiple-page    ("Single" or "Multiple" pages)
int     transparent color (color number for transparent color for GIF
                          export. -3=background, -2=None, -1=Default,
                          0-31 for standard colors or 32- for user colors)

# optional comment      (An optional set of comments may be here,
                          which are associated with the whole figure)

int     resolution coord_system (Fig units/inch and coordinate system:
                                1: origin at lower left corner (NOT USED)
                                2: upper left)
```

Fig_resolution is the resolution of the figure in the file.
Xfig will always write the file with a resolution of 1200ppi so it will scale the figure upon reading it in if its resolution is different from 1200ppi. Pixels are assumed to be square.

Note about metric units: To preserve a regular grid on the canvas the centimeter is defined to be 450 Fig units and not 472.4 (1200/2.54). For drawings done in metric units, fig2dev magnifies the output when exporting or printing to compensate for the difference (472.4/450). Also, if you make a drawing in one unit scale and switch to the other units in xfig, the drawing will be rescaled on the screen by xfig to remain consistent.

Xfig will read the orientation string and change the canvas to match either the Landscape or Portrait mode of the figure file.

The units specification is self-explanatory.

The coordinate_system variable is ignored - the origin is ALWAYS the upper-left corner.

```
** Coordinates are given in "fig_resolution" units.
** Line thicknesses are given in 1/80 inch (0.3175mm) or 1 screen pixel.
   When exporting to EPS, PostScript or any bitmap format (e.g. GIF), the
   line thickness is reduced to 1/160 inch (0.159mm) to "lighten" the look.
** dash-lengths/dot-gaps are given in 80-ths of an inch.
```

(3) The rest of the file contains various objects. An object can be one of six classes (or types).

- 0) Color pseudo-object.
- 1) Ellipse which is a generalization of circle.
- 2) Polyline which includes polygon and box.
- 3) Spline which includes closed/open approximated/interpolated/x-spline spline.
- 4) Text.
- 5) Arc.
- 6) Compound object which is composed of one or more objects.

In the following elaboration on object formats, every value of fig output are separated by blank characters or new line ('\n'). The value of the unused parameters will be -1.

Some fields are described as "enumeration type" or "bit vector"; the values which these fields can take are defined in the header file object.h. The pen_style field is unused. These values may be defined in some future version of Fig.

The two color fields (pen and fill; pen only, for texts) are defined as follows:

- 1 = Default
- 0 = Black
- 1 = Blue
- 2 = Green
- 3 = Cyan
- 4 = Red
- 5 = Magenta
- 6 = Yellow
- 7 = White
- 8-11 = four shades of blue (dark to lighter)
- 12-14 = three shades of green (dark to lighter)
- 15-17 = three shades of cyan (dark to lighter)
- 18-20 = three shades of red (dark to lighter)
- 21-23 = three shades of magenta (dark to lighter)
- 24-26 = three shades of brown (dark to lighter)
- 27-30 = four shades of pink (dark to lighter)
- 31 = Gold

values from 32 to 543 (512 total) are user colors and are defined in color pseudo-objects (type 0)

Your X server may limit the number of colors to something less than this, especially on a 8-bit PseudoColor visual, where the number of usable colors will be 256 minus the number of colors xfig preallocates for itself and the 32 standard colors (about 48).

For WHITE color, the area fill field is defined as follows:

- 1 = not filled
- 0 = black
- ... values from 1 to 19 are shades of grey, from darker to lighter
- 20 = white
- 21-40 not used
- 41-56 see patterns for colors, below

For BLACK or DEFAULT color, the area fill field is defined as follows:

- 1 = not filled
- 0 = white
- ... values from 1 to 19 are shades of grey, from lighter to darker
- 20 = black
- 21-40 not used
- 41-56 see patterns for colors, below

For all other colors, the area fill field is defined as follows:

- 1 = not filled
- 0 = black
- ... values from 1 to 19 are "shades" of the color, from darker to lighter.
 - A shade is defined as the color mixed with black
- 20 = full saturation of the color
- ... values from 21 to 39 are "tints" of the color from the color to white.
 - A tint is defined as the color mixed with white

- 40 = white
- 41 = 30 degree left diagonal pattern
- 42 = 30 degree right diagonal pattern
- 43 = 30 degree crosshatch
- 44 = 45 degree left diagonal pattern
- 45 = 45 degree right diagonal pattern
- 46 = 45 degree crosshatch
- 47 = horizontal bricks
- 48 = vertical bricks
- 49 = horizontal lines
- 50 = vertical lines
- 51 = crosshatch
- 52 = horizontal "shingles" skewed to the right
- 53 = horizontal "shingles" skewed to the left
- 54 = vertical "shingles" skewed one way
- 55 = vertical "shingles" skewed the other way
- 56 = fish scales
- 57 = small fish scales
- 58 = circles
- 59 = hexagons
- 60 = octagons
- 61 = horizontal "tire treads"
- 62 = vertical "tire treads"

The depth field is defined as follows:

0 ... 999 where larger value means object is deeper than (under)
objects with smaller depth

The line_style field is defined as follows:

- 1 = Default
- 0 = Solid
- 1 = Dashed
- 2 = Dotted
- 3 = Dash-dotted
- 4 = Dash-double-dotted
- 5 = Dash-triple-dotted

The style_val field is defined as the length, in 1/80 inches, of the on/off dashes for dashed lines, and the distance between the dots, in 1/80 inches, for dotted lines.

The join_style field is defined FOR LINES only as follows:

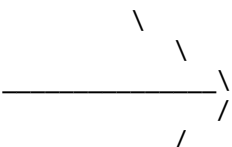
- 0 = Miter (the default in xfig 2.1 and earlier)
- 1 = Round
- 2 = Bevel

The cap_style field is defined FOR LINES, OPEN SPLINES and ARCS only as follows:

- 0 = Butt (the default in xfig 2.1 and earlier)
- 1 = Round
- 2 = Projecting

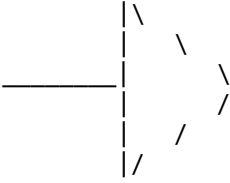
The arrow_type field is defined for LINES, ARCS and OPEN SPLINES only as follows:

- 0 = Stick-type (the default in xfig 2.1 and earlier)

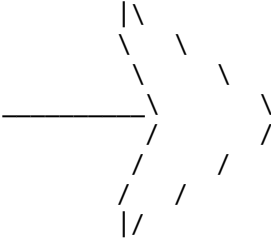


/

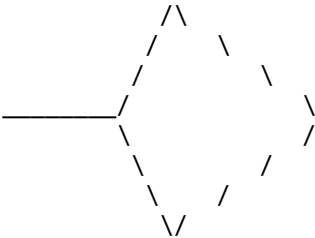
1 = Closed triangle:



2 = Closed with "indented" butt:



3 = Closed with "pointed" butt:



The arrow_style field is defined for LINES, ARCS and OPEN SPLINES only as follows:

- 0 = Hollow (actually filled with white)
- 1 = Filled with pen_color

(3.0) OBJECT DEFINITIONS:

===== (3.1) Color Pseudo-objects (user-defined colors) =====

This is used to define arbitrary colors beyond the 32 standard colors.
The color objects must be defined before any other Fig objects.

First line:

type	name	(brief description)
int	object_code	(always 0)
int	color_number	(color number, from 32-543 (512 total))
hex string	rgb values	(hexadecimal string describing red, green and blue values (e.g. #330099))

===== (3.2) ARC =====

First line:

type	name	(brief description)
int	object_code	(always 5)
int	sub_type	(1: open ended arc 2: pie-wedge (closed))

int	line_style	(enumeration type, solid, dash, dotted, etc.)
int	line_thickness	(1/80 inch)
int	pen_color	(enumeration type, pen color)
int	fill_color	(enumeration type, fill color)
int	depth	(enumeration type)
int	pen_style	(pen style, not used)
int	area_fill	(enumeration type, -1 = no fill)
float	style_val	(1/80 inch, specification for dash/dotted lines)
int	cap_style	(enumeration type)
int	direction	(0: clockwise, 1: counterclockwise)
int	forward_arrow	(0: no forward arrow, 1: on)
int	backward_arrow	(0: no backward arrow, 1: on)
float	center_x, center_y	(center of the arc)
int	x1, y1	(Fig units, the 1st point the user entered)
int	x2, y2	(Fig units, the 2nd point)
int	x3, y3	(Fig units, the last point)

Forward arrow line (Optional; absent if forward_arrow is 0):

type	name	(brief description)
int	arrow_type	(enumeration type)
int	arrow_style	(enumeration type)
float	arrow_thickness	(1/80 inch)
float	arrow_width	(Fig units)
float	arrow_height	(Fig units)

Backward arrow line (Optional; absent if backward_arrow is 0):

type	name	(brief description)
int	arrow_type	(enumeration type)
int	arrow_style	(enumeration type)
float	arrow_thickness	(1/80 inch)
float	arrow_width	(Fig units)
float	arrow_height	(Fig units)

=====

(3.3) COMPOUND

=====

A line with object code 6 signifies the start of a compound. There are four more numbers on this line which indicate the upper left corner and the lower right corner of the bounding box of this compound. A line with object code -6 signifies the end of the compound. Compound may be nested.

First line:

type	name	(brief description)
int	object_code	(always 6)
int	upperleft_corner_x	(Fig units)
int	upperleft_corner_y	(Fig units)
int	lowerright_corner_x	(Fig units)
int	lowerright_corner_y	(Fig units)

Subsequent lines:

```
objects
.
```

Last line:

```
-6
```

=====

(3.4) ELLIPSE

=====

First line:

type	name	(brief description)
----	----	-----
int	object_code	(always 1)
int	sub_type	(1: ellipse defined by radii 2: ellipse defined by diameters 3: circle defined by radius 4: circle defined by diameter)
int	line_style	(enumeration type, solid, dash, dotted, etc.)
int	thickness	(1/80 inch)
int	pen_color	(enumeration type, pen color)
int	fill_color	(enumeration type, fill color)
int	depth	(enumeration type)
int	pen_style	(pen style, not used)
int	area_fill	(enumeration type, -1 = no fill)
float	style_val	(1/80 inch, specification for dash/dotted lines)
int	direction	(always 1)
float	angle	(radians, the angle of the x-axis)
int	center_x, center_y	(Fig units)
int	radius_x, radius_y	(Fig units)
int	start_x, start_y	(Fig units; the 1st point entered)
int	end_x, end_y	(Fig units; the last point entered)

=====
(3.5) POLYLINE
=====

First line:

type	name	(brief description)
----	----	-----
int	object_code	(always 2)
int	sub_type	(1: polyline 2: box 3: polygon 4: arc-box) 5: imported-picture bounding-box)
int	line_style	(enumeration type, solid, dash, dotted, etc.)
int	thickness	(1/80 inch)
int	pen_color	(enumeration type, pen color)
int	fill_color	(enumeration type, fill color)
int	depth	(enumeration type)
int	pen_style	(pen style, not used)
int	area_fill	(enumeration type, -1 = no fill)
float	style_val	(1/80 inch, specification for dash/dotted lines)
int	join_style	(enumeration type)
int	cap_style	(enumeration type, only used for POLYLINE)
int	radius	(1/80 inch, radius of arc-boxes)
int	forward_arrow	(0: off, 1: on)
int	backward_arrow	(0: off, 1: on)
int	npoints	(number of points in line)

Forward arrow line: same as ARC object

Backward arrow line: same as ARC object

For picture (type 5) the following line follows:

type	name	(brief description)
----	----	-----
boolean	flipped	orientation = normal (0) or flipped (1)
char	file[]	name of picture file to import

Points line(s). The x,y coordinates follow, any number to a line, with as many lines as are necessary:

type	name	(brief description)
int	x1, y1	(Fig units)
int	x2, y2	(Fig units)
.	.	.
int	xnpoints ynpoints	(this will be the same as the 1st point for polygon and box)

(3.6) SPLINE

First line:

type	name	(brief description)
int	object_code	(always 3)
int	sub_type	(0: open approximated spline 1: closed approximated spline 2: open interpolated spline 3: closed interpolated spline 4: open x-spline 5: closed x-spline)
int	line_style	(enumeration type, solid, dash, dotted, etc.)
int	thickness	(1/80 inch)
int	pen_color	(enumeration type, pen color)
int	fill_color	(enumeration type, fill color)
int	depth	(enumeration type)
int	pen_style	(pen style, not used)
int	area_fill	(enumeration type, -1 = no fill)
float	style_val	(1/80 inch, specification for dash/dotted lines)
int	cap_style	(enumeration type, only used for open splines)
int	forward_arrow	(0: off, 1: on)
int	backward_arrow	(0: off, 1: on)
int	npoints	(number of control points in spline)

Forward arrow line: same as ARC object

Backward arrow line: same as ARC object

Points line: same as POLYLINE object

Control points line :

There is one shape factor for each point. For positive values of this factor, the spline is approximated at this point, for negative values the spline is interpolated at this point. The spline is always smooth in the neighbourhood of a control point, except when the value of the factor is 0 for which there is a first-order discontinuity (i.e., an angular point). Recommended values for this factor are 1.0 for an approximated spline, 0.0 for an angular point and -0.5 for interpolated splines.

The shape factor used here corresponds to the parameter s_k defined in section 4.1 of Blanc & Schlick (1995) for values greater than 0, for values smaller than 0 it corresponds to the negative of the parameter q defined in section 5.1 of Blanc & Schlick (1995).

(3.7) TEXT

type	name	(brief description)
int	object	(always 4)

int	sub_type	(0: Left justified 1: Center justified 2: Right justified)
int	color	(enumeration type)
int	depth	(enumeration type)
int	pen_style	(enumeration , not used)
int	font	(enumeration type)
float	font_size	(font size in points)
float	angle	(radians, the angle of the text)
int	font_flags	(bit vector)
float	height	(Fig units)
float	length	(Fig units)
int	x, y	(Fig units, coordinate of the origin of the string. If sub_type = 0, it is the lower left corner of the string. If sub_type = 1, it is the lower center. Otherwise it is the lower right corner of the string.)
char	string[]	(ASCII characters; starts after a blank character following the last number and ends before the sequence '\001'. This sequence is not part of the string. Characters above octal 177 are represented by \xxx where xxx is the octal value. This permits fig files to be edited with 7-bit editors and sent by e-mail without data loss. Note that the string may contain '\n'.)

The font_flags field is defined as follows:

Bit	Description
0	Rigid text (text doesn't scale when scaling compound objects)
1	Special text (for LaTeX)
2	PostScript font (otherwise LaTeX font is used)
3	Hidden text

The font field is defined as follows:

For font_flags bit 2 = 0 (LaTeX fonts):

0	Default font
1	Roman
2	Bold
3	Italic
4	Sans Serif
5	Typewriter

For font_flags bit 2 = 1 (PostScript fonts):

-1	Default font
0	Times Roman
1	Times Italic
2	Times Bold
3	Times Bold Italic
4	AvantGarde Book
5	AvantGarde Book Oblique
6	AvantGarde Demi
7	AvantGarde Demi Oblique
8	Bookman Light
9	Bookman Light Italic
10	Bookman Demi
11	Bookman Demi Italic
12	Courier

13	Courier Oblique
14	Courier Bold
15	Courier Bold Oblique
16	Helvetica
17	Helvetica Oblique
18	Helvetica Bold
19	Helvetica Bold Oblique
20	Helvetica Narrow
21	Helvetica Narrow Oblique
22	Helvetica Narrow Bold
23	Helvetica Narrow Bold Oblique
24	New Century Schoolbook Roman
25	New Century Schoolbook Italic
26	New Century Schoolbook Bold
27	New Century Schoolbook Bold Italic
28	Palatino Roman
29	Palatino Italic
30	Palatino Bold
31	Palatino Bold Italic
32	Symbol
33	Zapf Chancery Medium Italic
34	Zapf Dingbats